

Service Department Operational Optimization Simulation

Sankalp Amaravadi

CS 4632 Summer Section W01

July 20, 2026

GitHub: https://github.com/Leneord1/CS4632_Amaravadi_Sankalp

Abstract—This paper documents a discrete-event simulation of an automotive dealership service department. The model couples customer arrivals, advisor intake, technician and bay assignment, and parts inventory in a shared FIFO job queue. Arrivals follow a non-stationary Cox process; service times use experience-scaled Gamma draws; parts use a reorder-point inventory with stochastic lead time. Thirty-replication one-at-a-time sensitivity sweeps and five operational scenarios show that service rate and technician count dominate total wait, while parts stress inflates wait with little change in utilization. Extreme-condition and face checks support directional correctness; M/M/c output comparison is treated as a soft benchmark because advisors, Gamma service, and parts blocking are outside the analytical model. The Java 17 implementation, JavaFX UI, and batch analysis tools are publicly available on GitHub.

Index Terms—Discrete-event simulation, dealership service operations, queueing, sensitivity analysis, parts inventory, M/M/c benchmark.

CONTENTS

I	Introduction	2	IV	Implementation	3
I-A	Research Questions	2	IV-A	Language Choice	3
I-B	Project Objectives	2	IV-B	Environment	3
I-C	Report Organization	2	IV-C	As-Built Stack	3
II	Background and Literature Review	2	IV-D	Simulation Type	4
II-A	Background	2	IV-E	Core Algorithms and Data Structures	4
II-A1	Name	2	IV-F	Tracked Metrics	4
II-A2	Domain	2	IV-G	Challenges Overcome	4
II-A3	Problem Statement	2	V	Experimental Setup	4
II-A4	Scope	2	V-A	Baseline Parameters	4
II-A5	Assumptions	2	V-B	Sensitivity Design	4
II-B	Literature Review	2	V-C	Scenarios	4
II-C	Mathematical Models	2	V-D	Data Collection	4
II-C1	M/M/c Benchmark	2	VI	Results and Analysis	4
II-C2	Cox Arrivals	2	VI-A	Baseline Statistics	4
II-C3	Experience-Based Gamma Service	3	VI-B	Sensitivity Findings	4
II-C4	Parts Delay Decomposition	3	VI-C	Scenario Comparison	4
III	Model Design and Architecture	3	VII	Validation and Verification	4
III-A	Conceptual Model	3	VII-A	Face Validation	4
III-B	UML Diagrams	3	VII-B	Extreme Conditions	4
III-C	Class Diagram	3	VII-C	Parameter Validation	4
III-D	Activity Diagram	3	VII-D	Output Comparison to M/M/c	5
III-E	Key Design Decisions	3	VII-E	Limitations	5
			VIII	Discussion	5
			IX	Conclusion and Future Work	5
			Appendix A:	Simulation Parameters	5
			Appendix B:	Expected Outcomes Versus Observed Behavior	5
			References		5

I. INTRODUCTION

THIS project started as an idea while working in dealership service departments. During nearly six years across five dealerships and four brands, recurring bottlenecks appeared in technician scheduling, parts availability, repair bay utilization, and customer intake. Changing live workflow is costly; a simulation supports staffing and inventory decisions before real changes are made.

A. Research Questions

The study asks:

- (1) which controllable parameters most strongly affect total customer wait W_{total} and shop technician utilization.
- (2) how peak demand, understaffing, and parts stress alter wait, utilization, and throughput relative to a calibrated baseline.
- (3) whether simulated outputs remain directionally consistent with queueing theory and operational face validation.

B. Project Objectives

Objectives are to implement a complete discrete-event service-shop model, expose configurable parameters, collect wait, utilization, parts-delay, and throughput metrics, run replicated sensitivity and scenario experiments, and validate directional behavior against face checks, extremes, and an M/M/c soft benchmark.

C. Report Organization

Section II reviews background and related models. Section III presents architecture and UML. Section IV covers implementation. Sections V–VII report experimental design, results, and validation. Sections VIII–IX discuss implications and conclude.

II. BACKGROUND AND LITERATURE REVIEW

This project started as an idea while working in dealership service departments. During the nearly six years, five dealerships, and four brands, I noticed multiple bottlenecks in the department. The bottlenecks seemed to be large in technician scheduling, parts availability, repair bay utilization, and customer intake. In a place where changing the existing workflow could cost the company significant resources, having a simulation program will assist decision-making.

A. Background

1) *Name:* Service Department Operational Optimization Simulator

2) *Domain:* This project simulates the daily operation of an automotive dealership service department using a discrete-event simulation. Rather than modeling the physics behind vehicle dynamics, the simulation being developed focuses on operational workflows. This model looks at the relationship between customers, service advisors, technicians/repair(work) bays, and the parts department. The simulation outputs various metrics to identify bottlenecks, evaluate staffing decisions, and estimate the impact of the parts supply chain on shop performance.

3) *Problem Statement:* The key objective of this project is to analyze key metrics to find bottlenecks that increase customer wait times and decrease customer satisfaction and efficiency through reduced technician and bay utilization and productivity.

4) *Scope:* As the project is designed for an 8-week course, the scope will be limited. The core of the project will focus on developing the availability of parts, service advisor workflows, and technician workflows for a 30-bay service department. As customer payments do not have a major impact on overall workflow efficiency, they will not be included in this project's scope. Vehicle physics is too complex a task for the scope of this project.

5) *Assumptions:* Walk-in and appointment customers share one arrival stream; no-shows are omitted. Advisor intake time is constant. All technicians can perform all jobs; experience does not carry across replications; fatigue and breaks are out of scope. Each technician maps to one bay. Parts are not sold over the counter; replenishment uses one supplier with stochastic lead time. The job queue is strict FIFO. Financial and physics models are excluded.

B. Literature Review

Service operations simulations commonly rest on discrete-event foundations that advance state only at event epochs such as arrivals, assignments, and completions [5], [2]. Automotive service inventory work by Logachev *et al.* motivates explicit parts-state modeling rather than treating supply as instantaneous [1]. Non-stationary demand is represented with doubly stochastic (Cox) arrival processes [3]. Experience-dependent service duration follows technician routing literature with skill-scaled times [4]. The multi-server M/M/c model and Erlang-C waiting formulas supply analytical utilization $\rho = \lambda/(c\mu)$ and queue wait benchmarks used later as soft validation targets [5], [6]. Together these sources justify a DES with Cox arrivals, Gamma service scaled by experience, reorder-point parts inventory, and M/M/c comparison rather than pure analytical staffing alone.

C. Mathematical Models

1) *M/M/c Benchmark:* Arrivals are Poisson at rate λ ; exponential service rate μ per technician; c servers share a FIFO queue. Utilization is

$$\rho = \frac{\lambda}{c\mu}. \quad (1)$$

Erlang-C probability of wait $C(c, \lambda/\mu)$ yields expected queue wait

$$W_q = \frac{C(c, \lambda/\mu)}{c\mu - \lambda}. \quad (2)$$

Baseline illustration values $\lambda = 4$, $\mu = 2$, $c = 3$ give $\rho = 0.667$ and $W_q = 0.222$ h.

2) *Cox Arrivals:* Customer count in an interval follows a Poisson law with time-varying intensity, implemented as profile DEALERSHIP DAY.

3) *Experience-Based Gamma Service*: With normalized experience $e_i \in [0, 1]$ and coefficient α ,

$$f(e_i) = \frac{1}{1 + \alpha e_i}, \quad \mu_i = \mu_0 f(e_i), \quad T_i \sim \text{Gamma}(k, \theta_i), \quad (3)$$

4) *Parts Delay Decomposition*: Total delay uses

$$D_{\text{total}} = D_{\text{queue}} + D_{\text{parts}}, \quad (4)$$

and total customer wait

$$W_{\text{total}} = W_{\text{advisor}} + D_{\text{queue}} + D_{\text{parts}} + T_{\text{service}}. \quad (5)$$

III. MODEL DESIGN AND ARCHITECTURE

A. Conceptual Model

Vehicles progress through advisor intake, ticket creation, FIFO job-queue wait, bay occupation, optional parts blocking, repair, and departure. MetricsCollector records wait components, utilization, and throughput.

B. UML Diagrams

Two UML diagrams document the project, both authored in PlantUML and shown as: the class diagram and the activity diagram. The descriptions below reflect the implemented source rather than the original proposal sketch; notably the ticket entity is named `ServiceTicket`, and the parts logic is owned by a `PartsDepartment` class. Both diagrams are rendered from the PlantUML source and embedded in Fig. 1 and Fig. 2.

C. Class Diagram

The class diagram is organized around a discrete-event core. The `SimulationEngine` owns the event loop, scheduling `Event` records in a priority queue ordered by (time, sequence) and dispatching them by `EventType` (`ARRIVAL`, `INTAKE_COMPLETE`, `SERVICE_COMPLETE`, `PARTS_ARRIVAL`). The engine composes the `JobQueue`, `MetricsCollector`, and `PartsDepartment`.

The domain entities are `Customer`, `ServiceAdvisor`, `ServiceTicket` (with its `PartRequirement` list and `TicketStatus` of `WAITING`, `IN_PROGRESS`, `BLOCKED`, or `COMPLETE`), `Technician`, and `RepairBay`. A customer holds one ticket and is assigned to an advisor; the advisor creates the ticket; each technician maps to one bay, pulls from the shared `JobQueue`, and requests parts.

Parts handling is grouped in a dedicated package. `PartsDepartment` composes a `PartsInventory` of `Part` objects, tracks `PendingPartOrder` replenishments, blocks and releases tickets, and returns a `PartsFulfillmentResult` (status `FULFILLED` or `BLOCKED`). Relationships use composition for engine- and department-owned collaborators and dependencies for the transient request/block/release interactions.

D. Activity Diagram

The activity diagram traces one vehicle from arrival to departure across six swimlanes: Customer, Service Advisor, JobQueue, Technician, Parts Department, and MetricsCollector. A customer arrives under the Poisson/Cox process and waits until an advisor is available; the advisor intakes the customer, assesses vehicle condition, and creates a service ticket, which is enqueued in the job queue. A technician waits for availability, is assigned the ticket, and occupies a repair bay. Parts availability is a branch: if parts are unavailable the job is blocked and accrues D_{parts} while the technician pulls the next ticket and the parts department awaits fulfillment. Once work proceeds, the technician executes the repair over an experience-based service time, releases the bay, and returns to the queue. The advisor notifies the customer, who departs, and the MetricsCollector records W_{total} , utilization, and throughput. The flow repeats while events remain, then terminates.

E. Key Design Decisions

SimKit was replaced by a built-in priority-queue scheduler for maintainability. Target runtime is Java 17 rather than Java 22 for library maturity. Advisor experience effects on intake were dropped to match the constant-intake assumption. M1 “TBD” parameters were fixed in `simulation.properties` so experiments are reproducible.

IV. IMPLEMENTATION

A. Language Choice

Based on academic and personal experiences, Java is the language of choice. Since most of the components can be mapped to classes, a language that supports object-oriented programming would be preferred. Since there are quite a few parts of a dealership that can be added at a later point, having Java’s scalability would benefit the project significantly. Due to library support and experience, Java 22 and newer is preferred.

B. Environment

Development will be performed using JetBrains IntelliJ IDEA, JavaFX for visualization and user interaction, and SimKit for discrete-event simulation support. Source control and project documentation will be maintained through GitHub and Overleaf. AI-assisted development tools will be used throughout the implementation of this project. For documentation, ChatGPT is used for brainstorming [7] and assisting in generating citations aligned with IEEE standards from articles previously noted as being useful. Citation quality was verified by humans to see that all relevant information has been provided.

C. As-Built Stack

The shipped system uses Java 17, Maven, JUnit 5, JaCoCo, and JavaFX views (`LandingView`, `ConfigView`, `ResultsView`). Discrete-event scheduling is custom in `SimulationEngine`; SimKit is not required at runtime. Batch analysis uses `M4ExperimentRunner` with CSV/JSON export via `RunResultWriter`.

D. Simulation Type

The simulation will be implemented as a discrete-event simulation (DES). This approach is appropriate for modeling a service department, where system state changes are driven by distinct events such as customer arrivals, job assignments, parts requests, and job completions [2].

E. Core Algorithms and Data Structures

Event times are stored in a priority queue ordered by (time, sequence). Arrivals are generated by `CoxProcess` and `PiecewiseConstantArrivalRate`. Service durations use `ServiceTimeEquations` and `GammaDistribution`. Inventory uses reorder point r and quantity Q with stochastic lead time L . Validation compares shop utilization and queue delay to Erlang-C M/M/c outputs in `SimulationValidationMetrics`.

F. Tracked Metrics

The following performance metrics will be tracked throughout the simulation:

- Customer wait times
- Technician utilization
- Wait times for parts
- Jobs completed per day, reported both as a shop total and broken down per technician
- Job delays attributed to parts availability

G. Challenges Overcome

Non-stationary arrivals required a piecewise Cox intensity rather than a constant Poisson rate. Parts blocking had to release technicians to other tickets to avoid artificial idle lock. Strict M/M/c pass/fail flags never fire under realistic DES assumptions; validation therefore emphasizes directional and extreme checks.

V. EXPERIMENTAL SETUP

A. Baseline Parameters

Table I lists calibrated defaults from `simulation.properties`, used as the M4 baseline ($\lambda = 4/\text{h}$, $c = 3$, $\mu = 2/\text{tech/h}$, four advisors, horizon 10 h, Gamma $k = 4$, $\alpha = 1$, parts lead time $L = 2$ h). Each experiment uses $n = 30$ replications with seeds 42–71.

B. Sensitivity Design

One-at-a-time (OAT) sweeps vary arrival rate, technician count, service rate, advisor count, and parts lead time while holding other factors at baseline. Primary outputs are mean W_{total} , shop technician utilization, and jobs completed. Advisor and parts-lead sweeps do not always bracket the baseline point and are read as local trends.

C. Scenarios

Five scenarios are compared: Normal (baseline); Peak ($\lambda = 8$); Understaffed ($c = 2$, one advisor); Parts stress (longer lead time, tighter inventory); Ideal capacity ($c = 5$).

D. Data Collection

Each run writes time-series CSV (15-minute state snapshots), event CSV, summary JSON, and config JSON, indexed by `master_index.csv`. Aggregate M4 tables live under `documents/m4/`.

VI. RESULTS AND ANALYSIS

A. Baseline Statistics

At baseline, mean $W_{\text{total}} = 0.404$ h (95% CI [0.367, 0.441]), shop tech utilization 0.420 (CI [0.388, 0.452]), $D_{\text{parts}} = 0.022$ h, jobs completed 37.9/day, and $D_{\text{queue}} = 0.050$ h ($n = 30$).

B. Sensitivity Findings

Service rate μ and technician count c exert the strongest elasticities on wait. Raising μ from 2.0 to 2.5 cuts wait to 0.319 h (sensitivity -0.843). Moving c from 3 to 4 cuts wait to 0.362 h (sensitivity -0.313). Raising λ from 4 to 6 increases wait to 0.501 h (sensitivity 0.482). Parts lead time has a moderate wait effect; advisor count mainly shifts advisor wait, not total wait. Jobs completed stay near the arrival-driven ceiling except under saturation; small job-count differences across staffing levels fit replication noise.

C. Scenario Comparison

Table II and Fig. 5 summarize scenario means. Peak nearly doubles wait to 0.693 h and raises utilization to 0.788, with jobs rising to 73.7. Understaffing raises wait to 0.545 h and utilization to 0.616 while jobs stay near baseline. Parts stress inflates wait to 1.271 h with utilization near 0.428—phantom under-utilization from parts blocking. Ideal capacity ($c = 5$) cuts wait to 0.356 h and utilization to 0.264.

VII. VALIDATION AND VERIFICATION

Four approaches were applied: face validation, extreme-condition testing, parameter validation, and M/M/c output comparison [5].

A. Face Validation

Peak wait (0.693 h) exceeds baseline (0.404 h); understaffed wait (0.545 h) exceeds baseline; ideal-capacity wait (0.356 h) is below baseline; parts-stress D_{parts} (0.870 h) greatly exceeds baseline (0.022 h). Heuristic checks in the M4 validation table all pass.

B. Extreme Conditions

Low arrival yields utilization mean 0.024 (near idle). Overload yields wait mean 5.998 h and 195 jobs. One-technician wait mean is 2.613 h with utilization 1.323. Instant parts yields $D_{\text{parts}} = 0$.

C. Parameter Validation

Higher λ raises wait and utilization; faster μ and larger c reduce both. Directions match expected queueing behavior.

D. Output Comparison to M/M/c

Analytical $\rho = 0.667$ versus simulated shop util mean 0.420 (relative error mean 0.370, 95% CI [0.322, 0.417]). Mean $D_{\text{queue}} = 0.050$ h is below analytical $W_q = 0.222$ h. Zero of thirty baseline replications set `overallValid=true`. Disagreement is expected when advisors, Gamma service, non-stationary arrivals, and parts blocking dominate; the benchmark confirms mid-range utilization and sub-hour waits, not numerical equality.

E. Limitations

No warm-up deletion; metrics include the full 10 h horizon. Parameters are experience-based, not dealership-telemetry calibrated. Analytical M/M/c ignores advisors and parts. Relative error is a soft check, not a hard pass/fail of reality.

VIII. DISCUSSION

Sensitivity ranking answers the first research question: μ and c dominate wait; λ is next; parts lead time is moderate; advisors mainly affect intake wait. Scenario results answer the second: peak demand buys throughput at high wait and utilization; understaffing hurts wait even when jobs stay flat; parts stress separates supply failure from staffing failure; extra technicians beyond demand yield diminishing wait gains and spare capacity. Validation answers the third: directional and extreme checks hold; strict M/M/c agreement does not and should not be required for this DES.

Practical implication: managers should diagnose whether wait is staffing-driven or parts-driven before hiring. Surprising finding: completed jobs change little across several staffing levels at fixed λ and horizon, so wait and utilization are clearer decision signals than raw throughput on a fixed day length.

IX. CONCLUSION AND FUTURE WORK

This work delivered a complete DES of dealership service operations with Cox arrivals, experience-scaled Gamma service, parts inventory blocking, JavaFX configuration and results views, and replicated M4 analysis. Contributions include open, reproducible code; quantitative evidence that service rate and technician count dominate wait; and demonstration that parts stress can inflate wait without raising utilization.

Lessons learned include preferring a custom event scheduler over an unused library dependency, fixing parameters early for batch studies, and treating analytical queueing models as soft benchmarks when the DES is richer than M/M/c.

Future work includes warm-up deletion, dealership-calibrated arrivals and lead times, priority/warranty queues, advisor experience effects, multi-supplier parts, and financial cost layers linking under-utilization to dollars.

APPENDIX A SIMULATION PARAMETERS

Baseline defaults appear in Table I. Per-experiment M4 sweeps vary one factor at a time around that baseline; production single-replication defaults are overridden to 30 replications for statistical batches.

APPENDIX B

EXPECTED OUTCOMES VERSUS OBSERVED BEHAVIOR

Staffing shows diminishing returns: moving from three to five technicians shortens wait while utilization falls, matching spare capacity rather than added throughput. Parts delay decouples wait from utilization under parts stress. Peak $\lambda = 8$ absorbs volume at higher wait and utilization. Wait remains the most parameter-sensitive tracked metric, led by μ and c .

REFERENCES

- [1] M. Logachev, V. Bugrimov, and A. Kondrat'ev, "Simulation model of the car service inventory management system: input parameters," *Transportation Research Procedia*, vol. 63, pp. 2335–2340, 2022.
- [2] M. D. Rossetti, "2.5 Elements of Process-Oriented Simulation," in *Simulation Modeling and Arena*, GitHub.io, Nov. 10, 2021. [Online]. Available: <https://rossetti.github.io/RossettiArenaBook/ch2-ProcessModeling.html>. [Accessed: Jun. 09, 2026].
- [3] Y. Zheng and Z. Zheng, "Doubly stochastic generative arrivals modeling," *arXiv preprint arXiv:2012.13940*, 2020. [Online]. Available: <https://arxiv.org/abs/2012.13940>
- [4] X. Chen, B. W. Thomas, and M. Hewitt, "The technician routing problem with experience-based service times," *Omega*, vol. 61, pp. 49–61, 2016.
- [5] J. Banks, J. S. Carson II, B. L. Nelson, and D. M. Nicol, *Discrete-Event System Simulation*, 5th ed. Upper Saddle River, NJ: Pearson, 2013.
- [6] J. Pearce, "Erlang C formula – made simple with an easy worked example," *Call Centre Helper*, 2024. [Online]. Available: <https://www.callcentrehelper.com/erlang-c-formula-example-121281.htm>. [Accessed: Jun. 10, 2026].
- [7] OpenAI, "Automotive Service Center Simulation," *ChatGPT*, 2026. [Online]. Available: <https://chatgpt.com/share/6a2ad07f-8620-83ea-9266-f22b70fe0432>. [Accessed: Jun. 11, 2026].

LIST OF FIGURES

1	Class diagram rendered from documents/Classdiagram.puml.	7
2	Activity diagram rendered from documents/ActivityUML.puml.	8
3	Mean total customer wait versus arrival rate λ (baseline at $\lambda = 4$ included).	9
4	Mean total customer wait versus technician count c (baseline at $c = 3$ included).	10
5	Scenario comparison: mean total wait and shop technician utilization.	11